SEGGER SystemView ile kendi kaydını al

Senin projen için pratik rehber
STM32F407VG • FreeRTOS 10.3.1 • SystemView 4.10a
V1 / V2-QWB • S0 ve S5 • 6 Ekim 2026

Önce sorunun cevabı

Evet. Mevcut düzende önce karttan kısa bir kayıt alıyorsun. Hazırladığımız betik bunu **recording.SVDat** dosyasına dönüştürüyor. Ardından dosyayı SystemView'da açıp inceliyorsun. Kayıt zaten bilgisayardaysa kart bağlı olmadan da inceleyebilirsin.

Adım	Bu projede olan işlem
1. Çalıştır	PowerShell betiği senaryoyu başlatır ve kayıt süresini karta gönderir.
2. Kaydet	Firmware, görev ve kesme olaylarını kartın RAM'indeki RTT tamponuna yazar.
3. Dosyaya çıkar	Süre dolunca izleme durur. ST-LINK / OpenOCD RAM'i okur; betik izleme verisini ayırır.
4. İncele	recording.SVDat dosyasını SystemView'da açarsın.

İki ayrı iş var: firmware'e izleme desteği eklemek bir hazırlık işidir. Hazırlık tamamlandıktan sonra günlük kullanımın kayıt komutunu çalıştırmak, butona basmak ve dosyayı açmaktır.

Rehberin yolu

2 - Bağlantılar ve hazırlık
3 - S0 / S5 kaydı alma
4 - Kayıt dosyaları ve doğrulama
5 - SystemView ekranını kullanma
6 - Gecikmenin nedenini bulma
7 - V1 / V2 derleme ve karta yükleme
8 - Yeni projeye taşıma
9 - Sorun giderme ve başvuru

Bu rehber yerel projedeki betiklere göre hazırlanmıştır. Belgede verilen komutlar rehber hazırlanırken çalıştırılmadı; kartın yazılımı değiştirilmedi. Son bilinen yüklü sürüm V1'dir. Sonradan yükleme yaptıysan sürümü buna göre seç.

Bağlantıları hazırla

Bağlantı	Görevi
ST-LINK USB kablosu	Kart üzerine yazılım yükler. Kayıt sırasında SWD üzerinden RAM okumasını sağlar.
USB-TTL adaptörü	COM portundan senaryo komutlarını ve uygulamanın ölçüm verisini taşır. SystemView izleme akışı bu UART üzerinden gelmez.
USER butonu	Deneyde fiziksel olay üretir. Mavi USER butonunu kullan; siyah RESET butonu koşuyu yeniden başlatır.

1. Kablolar ve COM portu

Önceki çalışan bağlantıyı koru: adaptör **RX** → **PA2**, adaptör **TX** → **PA3**, **GND** → **GND**. UART mantık seviyesi karta uygun 3,3 V olmalıdır. Güç bağlantısını değiştirmene gerek yok.

Aygıt Yöneticisi'nde "Bağlantı Noktaları (COM ve LPT)" bölümünden adaptörün COM numarasına bak. Mevcut örnekler COM5 kullanıyor. Numara değişmişse komutta yalnızca **-Port COM5** kısmını değiştir.

2. Aynı bağlantıyı kullanan programları kapat

Seri terminali veya uygulamanın COM portunu açık tutan arayüzünü kapat. CubeIDE'da aktif debug oturumu ve başka bir ST-LINK bağlantısı bulunmasın. SystemView'da eski bir dosyanın açık kalması sorun değildir.

3. PowerShell'i proje klasöründe aç

```
cd D:\freertos-bootcamp\hafta-02
```

Komutların tamamında başlangıç klasörü burasıdır. V1 betiği de ortak bilgisayar ortamını kullandığı için **projects/v2/interface/.venv** içindeki Python'u çalıştırır; bu, yanlış sürümün kaydedildiği anlamına gelmez.

4. Sürümü eşleştir

Kart V1 ise capture-v1.ps1; kart V2-QWB ise capture-v2.ps1. Kayıt betiğinin kullandığı ELF, karttaki yazılımla aynı derlemeye ait olmalıdır. ELF dosyasından RAM sembollerinin adresleri okunur. Yalnızca betik adını değiştirerek karttaki sürümü değiştirmiş olmazsın.

Bu düzende harici J-Link kullanılmaz. F5 ile canlı kayıt başlatmak yerine aşağıdaki PowerShell akışı kullanılır. ST-LINK'in J-Link'e dönüştürülmesi gerekmez.

S0 ve S5 kaydını al

S0: telemetri yükü yok; buton yolunun temel gecikmesini gözlemlersin.
S5: 100 Hz telemetri yükü ve CPU işi etkin; görev beklemesi ve TX kuyruğunun etkisini gözlemlersin.

V1 karta yüklüyken

```
.\tools\capture-v1.ps1 -Scenario S0 -Windows 6 -WindowMs 1000 -Port COM5

.\tools\capture-v1.ps1 -Scenario S5 -Windows 8 -WindowMs 500 -Port COM5
```

V2-QWB karta yüklüyken

```
.\tools\capture-v2.ps1 -Scenario S0 -Windows 6 -WindowMs 1000 -Port COM5

.\tools\capture-v2.ps1 -Scenario S5 -Windows 8 -WindowMs 500 -Port COM5
```

Komutları tek tek çalıştır. Önce S0 koşusu tamamen bitsin; sonra S5'i başlat. Betik senaryoyu seçer, koşuyu başlatır ve hazırlık için yaklaşık 6 saniye bekler.

Komut çalışırken sen ne yapacaksın?

1. Terminalde **CAPTURE 1/6: press USER now** benzeri yazıyı bekle.
2. Mavi USER butonuna bas. Sonraki CAPTURE satırlarını takip ederek basışları tekrarla. Yaklaşık saniyede bir basışla başlayabilirsin.
3. Her pencere sonunda **Saved ... bytes** ve dosya yolu görünür.
4. Son pencereden sonra da bekle. Betik STOP ve ölçüm dökümünü tamamlar; V1'de son okuma yaklaşık 12 saniye sürebilir.
5. **OUTPUT=...** görünmesi ve komut satırının geri gelmesiyle koşu biter.

Parametre	Anlamı
-Windows 8	Sekiz ayrı kayıt penceresi. İşletim sisteminin adıyla ilgili değil.
-WindowMs 500	Her pencere 500 ms. Dışa aktarma sırasında pencereler arasında boşluk vardır.
-Scenario S5	Firmware'de tanımlı S5 yükünü seçer.

500 ms'lik pencereye her basış denk gelmeyebilir. BTN_ACCEPT içermeyen bir pencereyle buton gecikmesi çıkarma; başka pencereyi aç veya tekrar kayıt al. Uzun süre sınırsız basma: uygulamanın olay saklama kapasitesi sınırlıdır. Kayıt süresini büyütmek 64 KiB izleme tamponunu taşırabilir.

Dosyalar nerede, hangisi açılır?

Her koşu için yeni tarih-saat klasörü oluşur. Terminaldeki **Saved** satırı doğrudan kayıt dosyasını, **OUTPUT** satırı koşunun ana klasörünü gösterir.

```
traces/
  v1/                          # veya v2
    S5/                        # veya S0
      <tarih-saat>-observation/
        session.json
        dump.json
        serial_raw.bin
        window-01/
          recording.SVDat      # SystemView ile bunu aç
          trace.json
          ram.bin
          openocd.log
```

Dosya	Ne için kullanılır?
recording.SVDat	İkili SystemView olay kaydı; görev, kesme ve özel işaretler.
trace.json	Tek pencerenin tamamlanma, taşma, boyut ve hash bilgisi.
session.json	Koşunun senaryosu, derleme bilgisi, kayıt yolları, decoder_errors.
dump.json	Uygulamanın t0-t4 ölçümleri ve sayaçları. Kısa izleme pencerelerinin dışındaki olayları da içerebilir.
ram.bin / serial_raw.bin	İlki ham SRAM görüntüsü, ikincisi ham uygulama UART verisi. Bu projede doğrudan SystemView kaydı değiller.

Kaydı kullanmadan önce kontrol et

Pencerenin trace.json dosyasında **capture_complete: true**, **clean: true** ve **dropped_events: 0** beklenir. session.json içindeki decoder_errors değerini de kontrol et. “clean” yalnız izleme olay kaybını tarif eder; uygulamanın buton mesajı kaybetmediğini kanıtlamaz.

Dosya oluşmuş olması tek başına yeterli değildir. Betik hata verdiyse kısmi klasörü başarılı deney diye kullanma. Taşma varsa WindowMs değerini azaltıp yeni koşu al.

ram.bin dosyasının uzantısını .SVDat yapmak dönüştürme değildir. Bu projedeki betik ELF sembollerini kullanarak RTT kanal 1 verisini RAM görüntüsünden çıkarır. Ölçüm arşivinde firmware sürümünü, senaryoyu, ELF/build bilgisini ve tüm koşu klasörünü birlikte sakla.

SystemView'da dosyayı incele

Dosyayı aç

SystemView'ı başlat. Eğitim kullanımındaki lisans koşullarını kabul ediyorsan **Continue under SFL**, ardından varsa sistem bilgisi penceresinde **OK** seç. **File → Load Recording** (Ctrl+O) ile recording.SVDat dosyasını aç. [1]

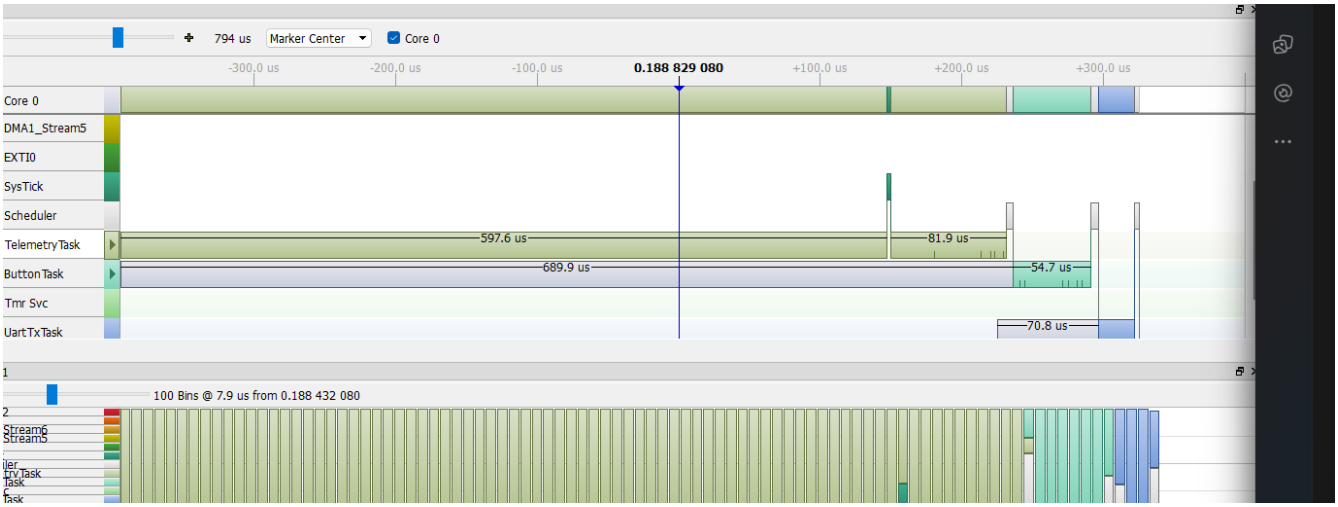
İstersen hazır betikle, dosya yolunu açıkça ver:

```
$trace = ".\traces\v1\S5\20261004-151332-observation"
.\tools\open-systemview.ps1 `
  -Trace "$trace\window-02\recording.SVDat"
```

Bu, mevcut arşivdeki V1 S5 örneğini açar. Yeni kaydın için terminalde yazan yolu kullan. open-systemview.ps1 parametresiz çalışırsa yalnız V2 altındaki en yeni temiz kaydı seçer; V1 için -Trace kullan.

Ekranı okuma sırası

Events List: BTN_ACCEPT olayını ve Detail içindeki run/id değerlerini bul. Satıra çift tıklayarak Timeline'ı aynı olaya getir. Listedeki # sıra numarası, uygulamadaki id ile aynı şey değildir.
Timeline: görevlerin hangi aralıkta çalıştığını izle.
CPU Load: görünen zaman aralığındaki yük dağılımını incele; yakınlaştırma değişince aralık da değişir.



Kendi V2 S5 ekranın: ButtonTask satırında gri çerçeveli 689,9 µs aralığı Ready beklemesi; turkuaz 54,7 µs aralığı çalışmadır. Yüksek öncelikli TelemetryTask CPU'yu kullanırken buton görevi bekler.

Gri = her zaman Blocked değildir. Ready, görevin çalışabilecek durumda CPU beklemesidir. Blocked, kuyruk, bildirim veya süre gibi bir koşulu beklemesidir. Soluk satır arka planını çalışma çubuğuyla karıştırma; Task Block olayını ve bekleme nedenini de kontrol et.

Task Run/Task Block görünmüyorsa View → Event Filter → Show Tasks filtresini kontrol et. Ctrl+Shift+Space filtreleri sıfırlar. Ctrl+G ile SystemView olay sıra numarasına gidebilirsin. [1]

“Neden gecikti?” sorusunu cevapla

Bir buton olayını sürüm + koşu klasörü + run + id ile eşleştir. Sadece id kullanma; kart yeniden başlayınca veya başka koşuda aynı sayı yeniden kullanılabilir.

İşaret	Ölçüm sınırı
BTN_ACCEPT / t0	EXTI içinde kabul edilen buton olayı.
BTN_TASK / t1	ButtonTask olayını kuyruktan alır. Task Run ile aynı sınır değildir.
BTN_READY / t2	Mesaj hazırlığı tamamlanır.
BTN_ENQUEUE	Mesaj TX kuyruğuna konur; doluluğu incele.
BTN_TX_START / t3	DMA gönderimini başlatma sınırı.
BTN_TX_TC / t4	UART aktarımının tamamlanma callback’i.
BTN_DROP	Mesaj düşmüştür; t3 ve t4 yoksa gecikmeyi sıfır yazma.

Dört aşamayı ayrı düşün

t1-t0: kesmeden olayı işleyene kadar bekleme.
t2-t1: mesaj hazırlama.
t3-t2: TX kuyruğu ve gönderim başlangıcı için bekleme.
t4-t3: aktarımın tamamlanması.
Toplam: t4-t0. DMA aktarım süresinin tamamını CPU çalışma süresi sayma.

Mevcut arşivle ilk alıştırman

V1 S5, **20261004-151332-observation/window-02** kaydını aç. Ctrl+G ile **#1732** olayına git: run=2, id=18 için BTN_ACCEPT. Aynı olay #1738’de BTN_TASK, #1739’da BTN_READY, #2765’te TX başlangıcı, #2799’da TX tamamlanmasıdır.

Bu olayda aşamalar **24 + 27 + 156449 + 5562 = 162062 µs**. Baskın süre mesaj hazırlığı değil, TX öncesi beklemedir. Kuyruk doluluğu ve TelemetryTask çalışması bunun nedenini incelemeni sağlar. Bunlar yeni kayıtta aynı sıra numaralarında olmak zorunda değildir.

Rapor yazarken

V1 S0 ile V2 S0’ı; V1 S5 ile V2 S5’i karşılaştır. S0’da baud değişimi hat süresini açıklar. S5’te daha hızlı UART, ayrı kuyruklar, BTN önce seçimi ve CPU işinin dilimlenmesi birlikte etkilidir. Mevcut deney bunların her birine ayrı kazanç yüzdesi vermez.

Rapor sayıları uygulamanın TIM5 t0-t4 değerlerinden gelir. SystemView zamanları farklı işaret sınırlarına ve izleme çağrısı zamanına aittir; birebir aynı sonucu bekleme. SWD okuma ve izleme kodunun etkisi sıfır kabul edilmemelidir. Kaynak: reports/V1_V2_Karsilastirma.md.

Sürüm değiştir veya kodunu yenile

Yalnız eski kayıtları incelemek için derleme ve yükleme gerekmez. Kod değiştirdiğinde veya V1'den V2'ye geçtiğinde **derle → karta yükle → eşleşen betikle kayıt al** sırasını uygula.

1. İzleme destekli derle

```
$py = ".\projects\v2\interface\.venv\Scripts\python.exe"

# V1
& $py .\projects\v1\firmware\build.py `
  --config Release --trace systemview

# V2-QWB
& $py .\projects\v2\firmware\build.py `
  --config Release --variant QWB --trace systemview
```

Sadece kullanacağın sürümü derle. Başarıyla tamamlandığını kontrol et. Oluşan ELF yolları:

```
# V1
projects/v1/firmware/build/v1/Release/systemview/Homework_001.elf

# V2-QWB
projects/v2/firmware/build/v2/QWB/Release/systemview/Homework_001.elf
```

2. Aynı ELF'i karta yükle

STM32CubeProgrammer'da ST-LINK bağlantısıyla karta bağlan. İlgili ELF dosyasını seçip programla, doğrulamanın başarılı olduğunu kontrol et, kartı resetleyip çalıştır. Kayıt almadan önce programlayıcının ST-LINK bağlantısını kapat. Sonra doğru capture betiğini kullan.

3. Yapılandırmayı koru

Mevcut entegrasyon **FreeRTOS 10.3.1** ve ona uyarlanmış V10 izleme kancalarını kullanır. Sırf SystemView kullanmak için RTOS sürümünü değiştirmen gerekmez. V1/V2 USART2 hızları sırasıyla 115200 ve 460800'dür; kayıt betikleri bunu kendileri ayarlar.

CubeMX ile .ioc dosyasından yeniden üretim yaparsan elle eklenmiş izleme kancalarını kaybetmediğini diff ile kontrol et. integrate_v1.py, integrate_v2.py ve instrument_app.py günlük kayıt komutları değildir; hazırlanmış projede tekrar çalıştırma.

Derleme çıktıları ve bilgisayardaki .venv GitHub'a eklenmez. Yeni bilgisayarda repo tek başına hazır çalışma ortamı değildir: STM32 araçları, Python ortamı, SystemView ve açıklama dosyaları ayrıca hazırlanır. Kayıt betikleri ST araçlarını C:/ST altında arar.

Yeni projende nasıl kullanırsın?

Bu projedeki kayıt araçları STM32F407 ve kendi UART protokolümüz için yazıldı. Aynı MCU ailesinde bile linker düzeni, UART protokolü ve semboller değişirse uyarlama gerekir.

Katman	Yapılacak iş
1. İzleme kaynakları	SystemView + RTT kaynaklarını ve kullandığın FreeRTOS sürümüne uygun adaptörü derlemeye ekle. Kütüphane sürümlerini sabitle.
2. RTOS olayları	Görev geçişi, Ready/Blocked ve ilgili RTOS API kancalarını doğru bağla. Aynı olayı iki kez kaydetme.
3. Zaman ve bellek	Zaman damgası kaynağını ve frekansını doğrula. RAM tamponunu, heap/stack ihtiyacını ve taşma davranışını bütçele.
4. Başlatma	İzleme yapılandırmasını başlat. Görev adları ve modül açıklamalarının her kayıtta mevcut olmasını sağla.
5. Uygulama işaretleri	İşin başlangıcı, kuyruğa giriş, işleme ve tamamlanma noktalarını adlandır. Kesme içinde bloklayan iş ekleme.
6. Bilgisayara aktarım	Mevcut ST-LINK snapshot yolunu uyarlayabilir veya uygun donanım/firmware ile başka bir recorder seçebilirsin.
7. Doğrulama	Önce tek görev ve düşük olay hızında dene. Bilinen bir periyodu doğrula; sonra yükü artırıp olay kaybını kontrol et.

Buradaki özel varsayımları bil

Mevcut betik 0x20000000 adresinden 128 KiB SRAM okur; 32 bit RTT tanımını, kanal 1'i ve uygulamaya özel app_trace_* sembollerini kullanır. Başka kartta bu adresleri ve belleğin büyüklüğünü aynen kopyalama.

Bizde tampon 64 KiB, zaman damgası DWT tabanlı ve CPU 168 MHz'dir. Başka MCU'da DWT bulunmayabilir veya düşük güç durumunda davranışı değişebilir. Uygulama ölçümlerindeki TIM5 kaynağı da ayrıca uyarlanmalıdır.

TRACE komutu, AppTrace yardımcıları ve capture betikleri projeye özeldir. SystemView kullanan her yazılım bu UART komutlarını otomatik olarak anlamaz. Canlı izlemeye geçiş için uygun recorder ve firmware taşıma katmanı gerekir; USB-TTL takmak tek başına yeterli değildir.

Yeni bilgisayarda: tools/SystemView-4.10a içindeki uygulamayı kur; docs/systemview-description içindeki SYSVIEW_*.txt dosyalarını uygulamanın Description klasörüne taşı. Ortak Python ortamını requirements.txt ile hazırla, STM32 toolchain/OpenOCD yollarını doğrula ve kaynaklardan yeniden derle. Bunlar mevcut yerel kurulumda zaten hazırlanmıştır.

Takılırsan buradan kontrol et

Belirti	İlk kontrol
COM açılamıyor	Doğru COM numarası mı? Seri terminal/arayüz portu kullanıyor mu?
Expected firmware missing	Karttaki V1/V2 ile betik eşleşiyor mu? UART çapraz bağlantı ve GND doğru mu?
OpenOCD / ST-LINK hatası	ST-LINK USB bağlı mı? Aktif debug/programlayıcı bağlantısı var mı? Pencerenin openocd.log dosyasını oku.
Invalid RTT / capture not frozen	ELF karttaki derleme ile aynı mı? İzleme destekli firmware yüklü mü? Kısmi kaydı rapora katma.
dropped_events sıfır değil	Kayıt penceresini kısalt. Tekrar al; eksik olaylardan gecikme sonucu çıkarma.
Task Run/Block görünmüyor	Görev filtresini aç, filtreleri sıfırla. Uygulama işaretleri ile RTOS olaylarını ayır.
BTN_ACCEPT yok	Buton basışı pencereye denk gelmemiş olabilir. Sonraki pencereyi aç veya yeniden kayıt al.
Yanlış/eski kayıt açılıyor	open-systemview.ps1 komutuna -Trace ile tam dosya yolunu ver. Başlıktaki sürümü kontrol et.
Olay isimleri sayısal/anlamsız	SYSVIEW_FreeRTOS.txt ve doğru V1/V2 modül açıklaması SystemView Description klasöründe mi?

Her koşudan sonra üç şey sakla

1. Kayıt ve ölçüm dosyaları.
2. Sürüm, senaryo, derleme ve kullandığın komut.
3. “Şu olaydan sonra şu görev bekledi; şu kanıt nedeniyle” biçiminde kısa gözlem ve ekran görüntüsü.

Kaynaklar ve devam noktaları

[1] SEGGER SystemView User Guide, 4.10: temel dosya açma, filtre ve arayüz işlemleri. Erişim: 6 Ekim 2026. https://doc.segger.com/UM08027_SystemView.html

Projeye özgü komut ve dosya davranışı: tools/capture-v1.ps1, capture-v2.ps1, capture_v1.py, capture_v2.py ve open-systemview.ps1. Deney yorumu: reports/V1_S0_S5.md, V2_S0_S5.md ve V1_V2_Karsilastirma.md. Bunların tamamı hafta-02 altındadır.

GitHub: github.com/furkankara32/freertos-bootcamp • Bu rehber hazır yerel ortamın kullanımını anlatır; yeni makinede bağımlılıkların kurulması ayrıca gerekir.